

**BERUFLICHES GYMNASIUM  
TECHNIK**  
**JOHANN-CONRAD-SCHLAUN-BERUFSSKOLLEG**



**Dokumentation**

**Go-Kart  
Blinker Lenkrad Anzeige**

Dean Schneider

|                  |                                           |
|------------------|-------------------------------------------|
| Auftraggeber:    | Michael Schmidt                           |
| Projektleitung:  | Henry Otto                                |
| Projektnummer:   | JCS-BK_1091_GO-KART                       |
| Jahrgang:        | GYT26                                     |
| Fach:            | Projekt mit Elektrotechnik und Informatik |
| Verfasser:       | Dean Schneider                            |
| Projektstart:    | 1. September 2025                         |
| Projektende:     | 20. März 2026                             |
| Letzte Änderung: | 23. Januar 2026                           |

# **Abstract**

[englische version]

In den vergangen Jahren haben immer wieder Gruppen der gymnasialen Oberstufe aus dem Technikbereich an dem Kettcar gearbeitet. Das Ziel ist mehrere Module eines KFZ im kleinen Aufzubauen (siehe 2.1 Team, S. 4). Dabei kommunizieren alle Module über CAN-Bus. Dies ist die Dokumentation zu dem Projekt welches die Lichtanlage, Lenkrad und die Tachoanzeige (Cockpit-Display) behandelt.

[aktueller Stand]

[was muss noch gemacht werden]

# Inhaltsverzeichnis

|                                                                |    |
|----------------------------------------------------------------|----|
| Abstract.....                                                  | 1  |
| 1 Einleitung.....                                              | 3  |
| 1.1 Code Konventionen.....                                     | 3  |
| 2 Ergebnis.....                                                | 4  |
| 2.1 Team.....                                                  | 4  |
| 2.2 Kommunikation zwischen den Projekten.....                  | 4  |
| 2.3 Dateistruktur.....                                         | 5  |
| 2.4 Cockpit-Display (Tacho).....                               | 6  |
| 2.5 Bluetooth und UART.....                                    | 8  |
| 2.6 CAN-Bus.....                                               | 8  |
| 2.7 Debug-Display (OLED-Display).....                          | 9  |
| 2.8 Mikrocontroller.....                                       | 10 |
| 2.8.1 ATmega16M1.....                                          | 10 |
| 2.8.2 AT90CAN32.....                                           | 11 |
| 3 Entwicklung.....                                             | 12 |
| 3.1 Programmierer.....                                         | 12 |
| 3.2 Erste Schritte mit Masterplatine.....                      | 13 |
| 3.3 Cockpit Display (Tacho).....                               | 14 |
| 3.3.1 Erstes Display.....                                      | 14 |
| 3.4 Bluetooth-Modul.....                                       | 15 |
| 3.4.1 Test und konfigurieren mit USB-zu-Seriell-Konverter..... | 15 |
| 3.4.2 Schaltung mit Mikrocontroller.....                       | 17 |
| 4 Grundlagen.....                                              | 19 |
| 4.1 Kompilieren und Hochladen.....                             | 19 |
| 4.2 UART Übertragung.....                                      | 20 |
| 4.2.1 Text übertragen.....                                     | 20 |
| 4.2.2 Rohdaten übertragen.....                                 | 20 |
| 4.3 CAN Sniffing (PCAN-USB).....                               | 21 |
| 4.3.1 Empfangen.....                                           | 21 |
| 4.3.2 Senden.....                                              | 21 |
| 4.3.3 Fehlerbehebung.....                                      | 21 |
| 4.4 Bluetooth Übertragung.....                                 | 22 |
| 5 Eidesstattliche Erklärung.....                               | 23 |
| 6 Anhang.....                                                  | 24 |
| 6.1 Projekte der Vorgänger.....                                | 24 |
| 6.2 Tagesziele.....                                            | 25 |
| 6.3 Meilensteine.....                                          | 27 |
| 6.4 Terminplan.....                                            | 28 |
| 6.5 Abbildungsverzeichnis.....                                 | 33 |
| 6.6 Schaltplanausschnitte.....                                 | 33 |
| 6.7 Tabellenverzeichnis.....                                   | 33 |

# 1 Einleitung

Diese Dokumentation richtet sich an Personen, die dieses Projekt weiterführen oder warten möchten. Die Themen sind nach ihrer Priorität für das Nachschlagen geordnet. Das bedeutet, dass zunächst der aktuelle Aufbau (das **Ergebnis**) beschrieben wird, anschließend die **Entwicklung** hin zum aktuellen Aufbau und schließlich die **Grundlagen** zur Bedienung der Software.

Gesamte Anleitung für Linux um genau zu sein für die Arch Distribution.

MCU (Microcontroller Unit) entspricht Mikrocontroller

Das hier genannte **Cockpit-Display** wird in anderen Dokumentationen Tacho, Tachodisplay oder einfach nur Display genannt. Das OLED-Display wird **Debug-Display** genannt, um Verwechslung zwischen den beiden Displays zu vermeiden.

Hexadezimalzahlen werden durch 0x gekennzeichnet (z. B. 0xFF oder 0x003B).

Schaltplanausschnitte sind ausschließlich aus dem Schaltplan für dieses Projekt.

[pflichtenheft vorgaben]

## 1.1 Code Konventionen

- Einheitliche Einrückung mit 4 Leerzeichen (keine Tabs)
- Maximale Zeilenlänge von 80 Zeichen
- Variablen und Funktionen: snake\_case
- Konstanten: UPPER\_CASE
- Aussagekräftige Namen statt Abkürzungen (calculate\_total statt calc)
- Kommentare richten sich nach dem Google Style C++ Guide (<https://google.github.io/styleguide/cppguide.html#Comments>)

## 2 Ergebnis

### 2.1 Team

Dieses Projekt wird in enger Zusammenarbeit mit dem Team realisiert. So muss die Kommunikation abgesprochen werden. Die Verteilung der Aufgaben sieht wie folgt aus:

| Name                | Aufgabe                                     |
|---------------------|---------------------------------------------|
| Henry Otto          | Motoransteuerung und Auslesung der Pedalen  |
| Iuri Merklein Muniz | Elektrische Bremse und Gangschaltung        |
| Tammo Lohe          | Ladeelektronik, Batterie und Solar          |
| Dean Schneider      | Lenkrad, Cockpitdisplay und die Beleuchtung |

Tabelle 1: Team

### 2.2 Kommunikation zwischen den Projekten

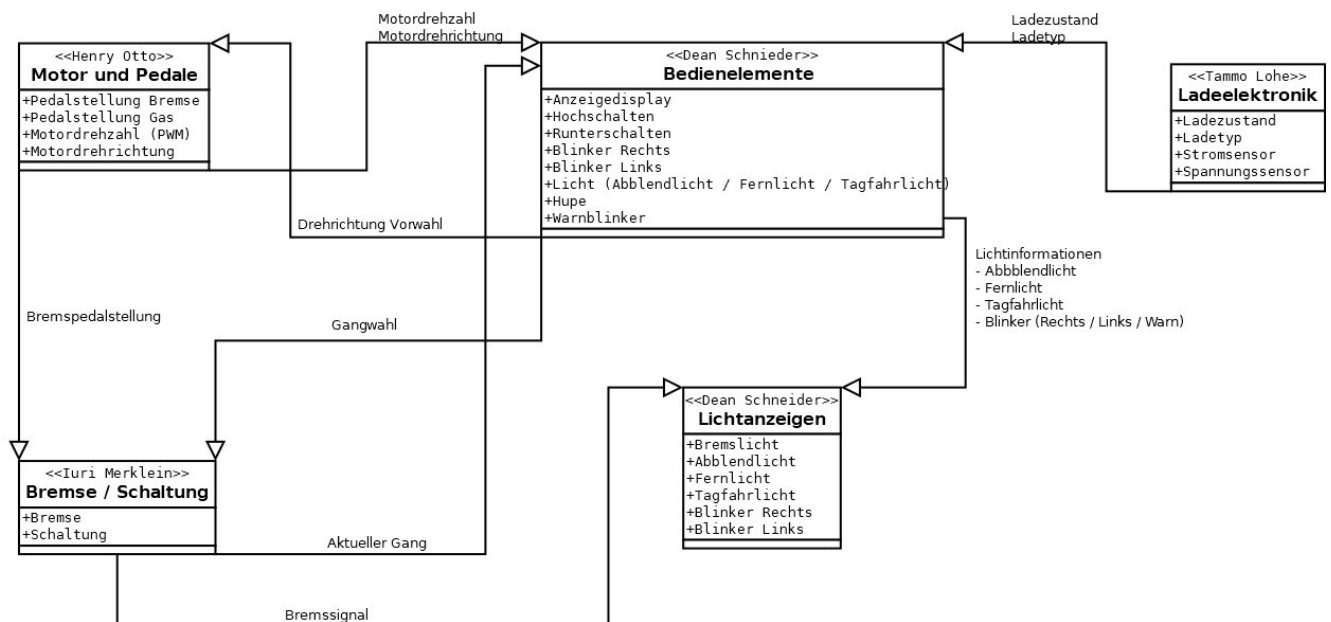
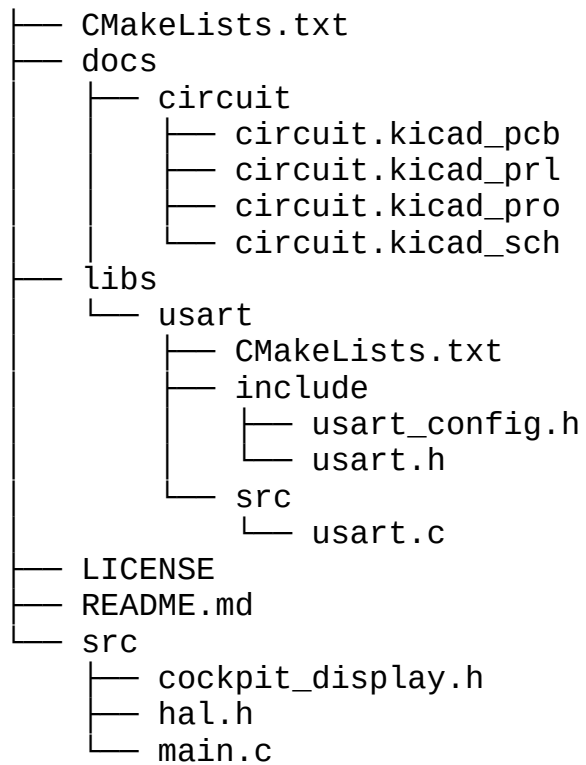


Abbildung 1: Kommunikationsdiagramm

[erklärung zur Projekt kommunikation]

## 2.3 Dateistruktur

Der Quellcode und weitere Dateien des Projektes sollten von dem Projektleiter bereitgestellt werden.



## 2.4 Cockpit-Display (Tacho)



Abbildung 2: Cockpit-Display (NX4024K032)

Das Display ist von Nextion und wird mit der Nextion typischen Software programmiert/eingestellt. Auf dem Glass befindet sich ein Riss von einem Vorgänger. Christian Barth (JCS-BK\_1046) hat das Display gestaltet. Um die Symbole oder Zahlen zu ändern werden Befehle seriell mit einer Baudrate von 9600 bit/s gesendet (siehe 4.2.2 Rohdaten übertragen, S. 20).

| Funktion                             | ID                               | Wertebereich                 |              |                                                   |              |   |
|--------------------------------------|----------------------------------|------------------------------|--------------|---------------------------------------------------|--------------|---|
| linker Blinker                       | bL.pic                           | ausgeschalten                | 0            | angeschalten                                      | 1            |   |
| rechter Blinker                      | bR.pic                           | ausgeschalten                | 2            | angeschalten                                      | 3            |   |
| Warnblinker                          | warnB.pic                        | ausgeschalten                | 15           | angeschalten                                      | 16           |   |
| Scheinwerfer                         | light.pic                        | Abblendlicht                 | 17           | Fernlicht                                         | 18           |   |
|                                      |                                  | Standlicht                   | 19           |                                                   |              |   |
| Drehzahl<br>(Umdrehungen pro Minute) | rpm1.pic<br>rpm2.pic<br>rpm3.pic | Hunderter<br>Zehner<br>Einer | 4 – 13       | Index 4 entspricht 0 und<br>Index 13 entspricht 9 |              |   |
| Geschwindigkeit (km/h)               | speed1.pic<br>speed2.pic         | Zehner<br>Einer              |              |                                                   |              |   |
| gefahrene Distanz                    | dis1.pic<br>dis2.pic<br>dis3.pic | Hunderter<br>Zehner<br>Einer |              |                                                   |              |   |
|                                      | eingelegter Gang                 | gear.pic                     | Neutralgang  | 14                                                | erster Gang  | 5 |
|                                      |                                  |                              | zweiter Gang | 6                                                 | dritter Gang | 7 |
| Batterie-Zustand                     | battery.pic                      | leer                         | 20           | 1/4                                               | 21           |   |
|                                      |                                  | 2/4                          | 22           | 3/4                                               | 23           |   |
|                                      |                                  | 4/4                          | 24           | wird geladen                                      | 25           |   |
| Gaspedal                             | throttle.val                     | 0 – 100                      |              |                                                   |              |   |
| Bremspedal                           | brake.val                        | 100 – 0                      |              |                                                   |              |   |

Tabelle 2: Befehle für das Cockpit-Display

So kann man zum Beispiel mit dieser Nachricht an das Display den linken Blinker einschalten:

```
$ stty -F /dev/ttyUSB0 9600
$ printf 'bL.pic=1\xFF\xFF\xFF' > /dev/ttyUSB0
```

Erklärung dieser Befehle lässt sich unter *4.2.2 Rohdaten übertragen* (siehe S. 20) finden.

Mit C-Code könnte das dann so aussehen:

```
uart_puts("bL.pic=1\xFF\xFF\xFF");
```



## 2.5 Bluetooth und UART

In diesem Projekt wird die Bibliothek *AVR-UART-lib* von jnk0le (<https://github.com/jnk0le/AVR-UART-lib>) verwendet.

|                              |                 |
|------------------------------|-----------------|
| Name des Bluetoothgeräts     | Blinker_Anzeige |
| Passwort des Bluetoothgeräts | 1234            |

## 2.6 CAN-Bus

[canbus erklärung]

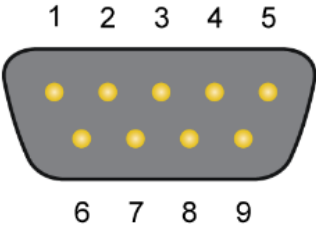
| Pin | Assignment        | D-Sub plug                                                                          |
|-----|-------------------|-------------------------------------------------------------------------------------|
| 1   | CAN_V+ (optional) |  |
| 2   | CAN_Low           |                                                                                     |
| 3   | CAN_GND           |                                                                                     |
| 4   | Not connected     |                                                                                     |
| 5   | Not connected     |                                                                                     |
| 6   | CAN_GND           |                                                                                     |
| 7   | CAN_High          |                                                                                     |
| 8   | Not connected     |                                                                                     |
| 9   | CAN_V+ (optional) |                                                                                     |

Abbildung 3: männlicher CAN D-Sub Anschluss nach CiA® 106  
(Quelle: PCAN-USB Anleitung)

|                  |                      |
|------------------|----------------------|
| CAN-Bus Baudrate | 125 kbit pro Sekunde |
|------------------|----------------------|

## 2.7 Debug-Display (OLED-Display)

Das verwendete Display ist das typische Arduino OLED-Display mit dem eingebauten SSD1306 Chip. Dieser wird über I<sup>2</sup>C angesteuert.

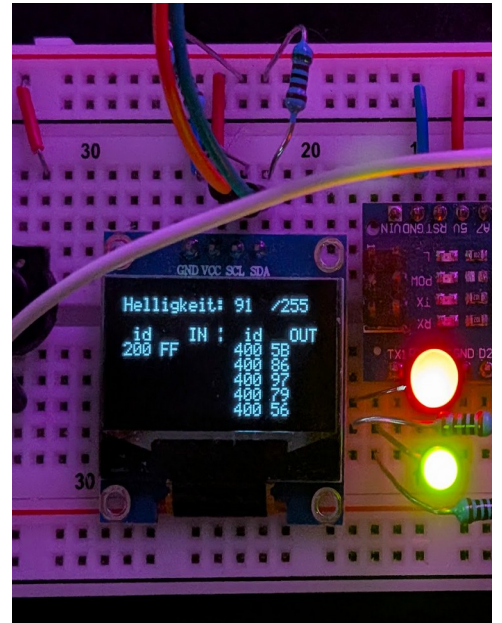


Abbildung 4: Debug-Display  
(OLED-Display mit SSD1306)

## 2.8 Mikrocontroller

### 2.8.1 ATmega16M1

Für die Lichtanlage wird der ATmega16M1 Mikrocontroller verwendet.

[features]

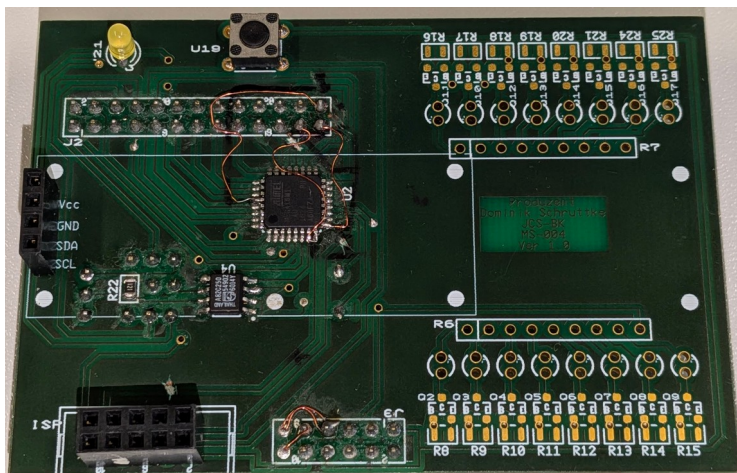


Abbildung 6: Oberseite des ATmega16M1

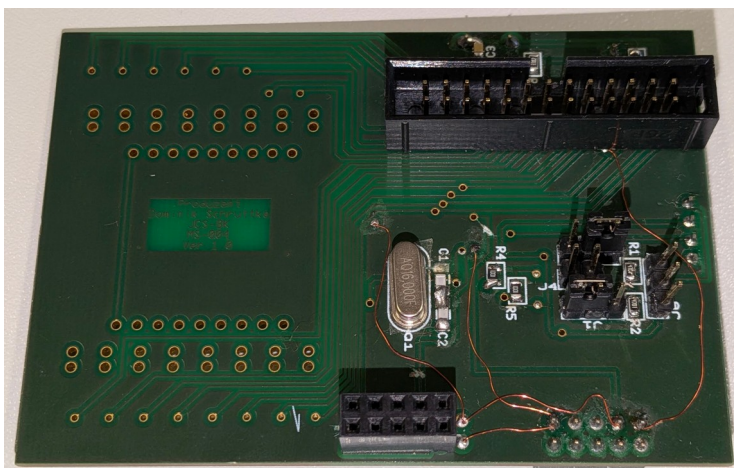


Abbildung 7: Unterseite des ATmega16M1

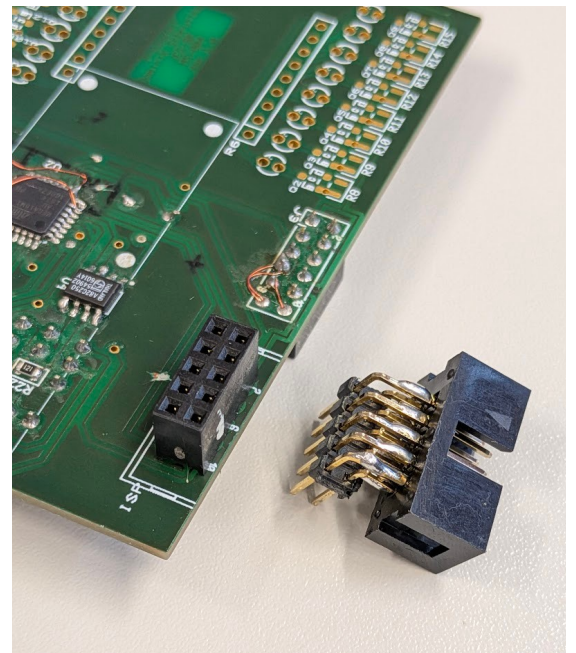


Abbildung 5: Stecker zum Programmieren des ATmega16M1

## 2.8.2 AT90CAN32

Die Platine mit dem AT90CAN32 Mikrocontroller wird für das Cockpit-Display und das Lenkrad verwendet. Da es die zentrale Steuereinheit für dieses Projekt ist, wird sie auch als Masterplatine bezeichnet. Das Projekt JCS-BK MS-001 enthält Schaltpläne zur Masterplatine. Pläne zur Addon-Platine, dass heißt dort wo die serielle und ISP Verbindung angeschlossen wird, lassen sich unter JCS-BK MS-003 finden.

Die wichtigsten Vorteile des AT90CAN32 gegenüber dem ATmega16M1 für dieses Projekt sind:

- zwei USART-Schnittstellen gegenüber einer beim ATmega16M1
- stärkerer CAN-Controller mit mehr Funktionen und Message Objects:
  - AT90CAN32: 15 Full Message Objects
  - ATmega16M1: 6 Message Objects
- Hardware I2C-Controller

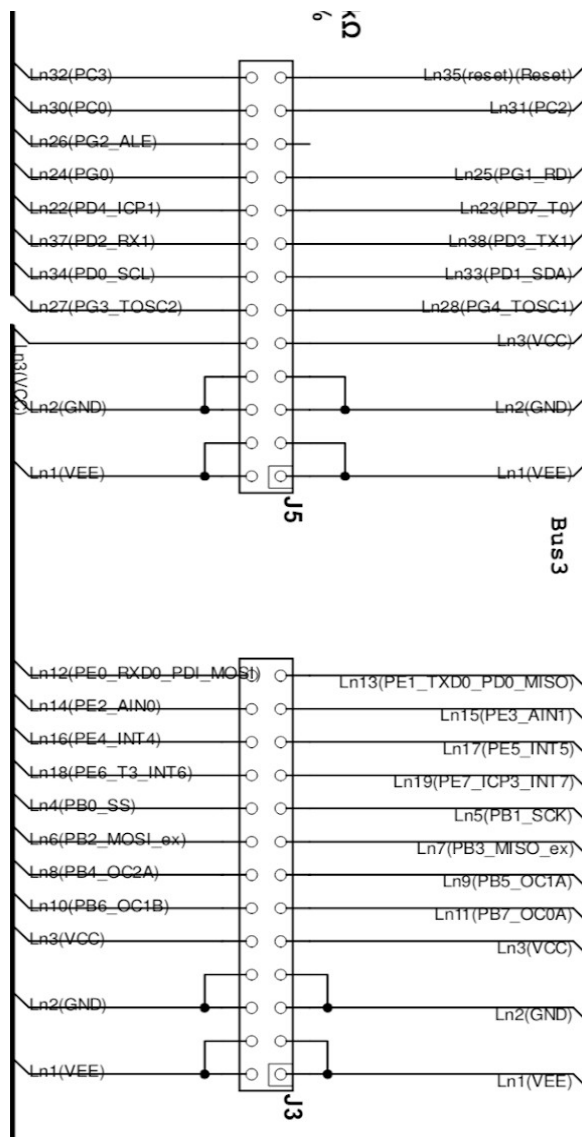


Abbildung 8: Anschlüsse des AT90CAN32  
(Quelle: Schaltplanausschnitt von MS-001)

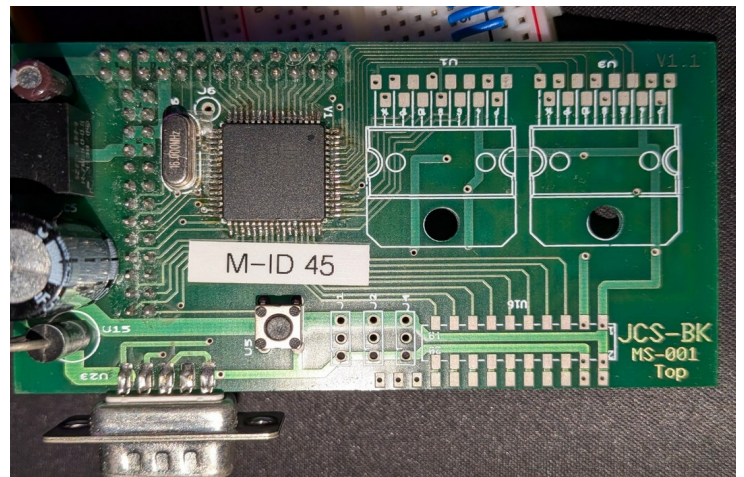


Abbildung 9: Masterplatine mit AT90CAN32 (MS-001)

## 3 Entwicklung

In diesem Kapitel wird der Prozess wie es Ergebnis kam dargestellt. Der aktuelle Stand des Projektes befindet sich im Kapitel 2 *Ergebnis* (Beginn S. 4).

### 3.1 Programmierer

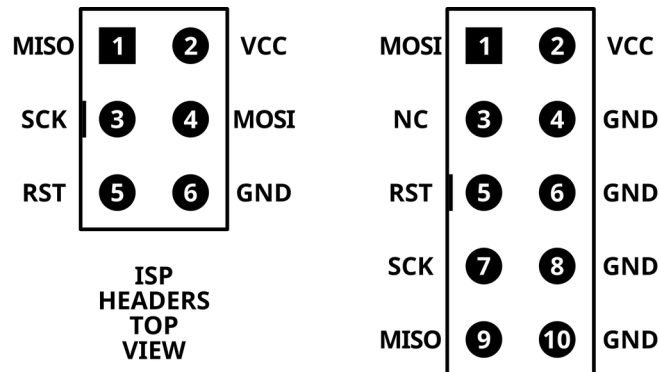


Abbildung 10: ISP Pinbelegung (Quelle: wikipedia.org von osiixy erstellt)

Ein Arduino besitzt einen Bootloader welcher den Eingang des UART in den Speicher schreibt, er programmiert sich also selbst. Allerdings besitzt der AVR MCU (AT90CAN32) von der Masterplatine keinen Bootloader, er wird über ISP (In-System Programming) programmiert.

Beim Kauf eines Programmierers ist darauf zu achten, dass er eine Open-Source Firmware hat, zum Beispiel USBasp.

Falls man nicht weiß welche Programmierer-Firmware man hat, kann man diese über die ID herausfinden:

```
$ lsusb
```

Zum Hochladen wird avrdude verwendet dasselbe Tool wie Microchip Studio verwendet. Als Erstes muss avrdude installiert werden. Zum Beispiel auf Arch mit:

```
$ sudo pacman -S avrdude
```

Anschließend kann man sich mit diesem Befehl alle voreingestellte Programmierer-Konfigurationen ausgeben lassen:

```
$ avrdude -c ?
```



## 3.2 Erste Schritte mit Masterplatine

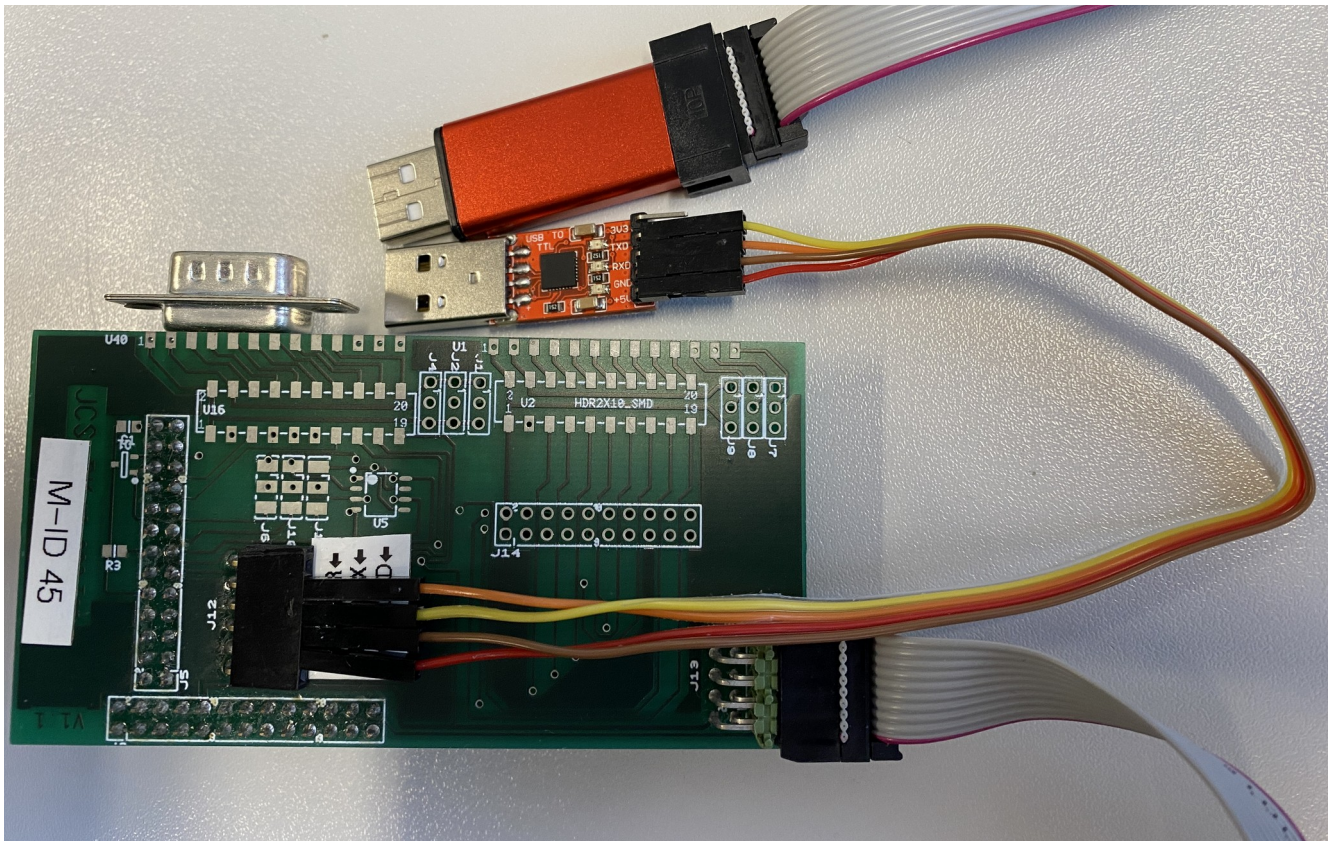


Abbildung 11: Masterplatine mit serieller und ISP Verbindung

Bevor man komplexe Programme programmiert sollte man erstmal die Programmier-Schnittstelle überprüfen. In diesem Beispiel wird TX zwischen High und Low gewechselt. Mein USB-zu-Seriell Gerät hat eine LED für RX, darüber kann ich den Spannungswechsel sehen. Kurz gesagt wird eine blinkende LED zum Feststellen der Funktion verwendet.

**Warnung:** In Abbildung 11 ist der Aufbau zu sehen. Es ist wichtig zu überprüfen ob der Programmierer eine Spannung liefert (z. B. 5V), denn dann sollte die Spannungsversorgung des USB-zu-Seriell Adapters von der Masterplatine getrennt werden. Oft ist dies nicht nötig, es reduziert jedoch das Risiko eines Schadens durch Ausgleich-Ströme.

Hier ist der Code zum Blinken lassen der RX LED (siehe 4.1 Kompilieren und Hochladen, S. 19):

```
#include <avr/io.h>
#include <util/delay.h>

static const uint8_t TXD1 = PD3;

int main(void) {
    DDRD = 1 << TXD1;

    while (1) {
        PORTD ^= 1 << TXD1;
        _delay_ms(500);
    }
}
```

## 3.3 Cockpit Display (Tacho)

### 3.3.1 Erstes Display

Eigentlich sollte das NX8048K070\_011C Display von Nextion verwendet werden. Allerdings war ich direkt zu Beginn des Projektes unaufmerksam und habe ausversehen an das 5V Display 14V angeschlossen, das Display leuchtete bis die Spannungsversorgung entfernt wurde. Der Spannungsregler des Displays ist beschädigt und eventuell auch weitere Bauteile. Diese müssten ausgetauscht werden, wobei sich die Frage stellt ob sich der Zeitaufwand lohnt, insbesondere da es sich um SMD Bauteile handelt und man nicht eindeutig die Bauteilbezeichnung lesen kann.

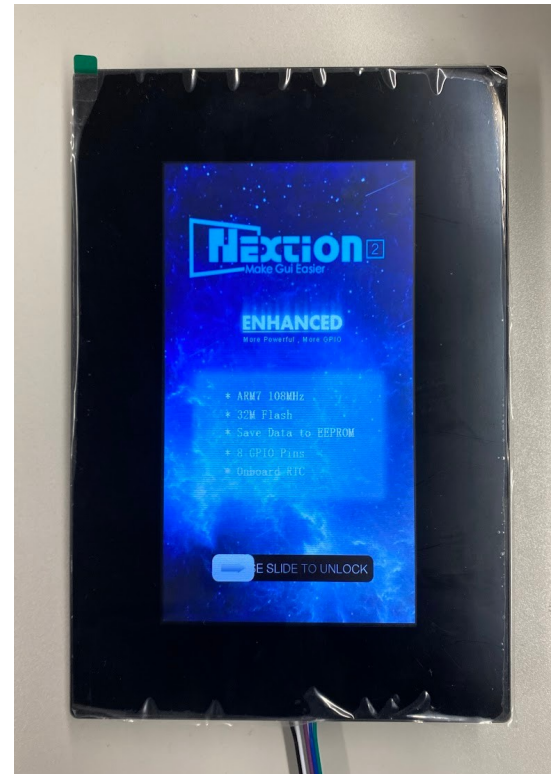


Abbildung 12: Nextion Display (NX8048K070\_011C)

|     |                |                                 |                                                                                       |                                                         |
|-----|----------------|---------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------|
| V6H | XC6702D771PR-G | Torex Semiconductor<br>SOT-89-5 |  | Linear voltage regulator IC<br>LDO, 7.7V±1%, 300mA, +CE |
| V6H | XC6702D771ER-G | Torex Semiconductor<br>USP-6C   |  | Linear voltage regulator IC<br>LDO, 7.7V±1%, 300mA, +CE |

Abbildung 13: Ersatz Bauteil suche für Spannungsregler (<https://smd.yooneed.one>)

Nun muss das vorherige Display, welches allerdings von JCS-BK\_1046 bereits gestaltet und eingestellt wurde, verwendet werden.

### 3.4 Bluetooth-Modul

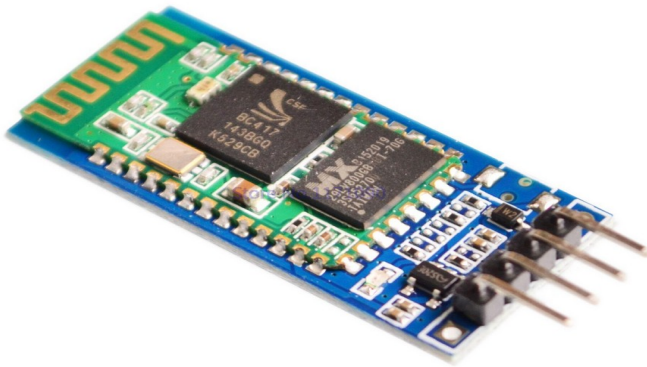


Abbildung 14: Bluetooth-Modul mit HC-06 Oberseite  
(Quelle: roboter-bausatz.de)

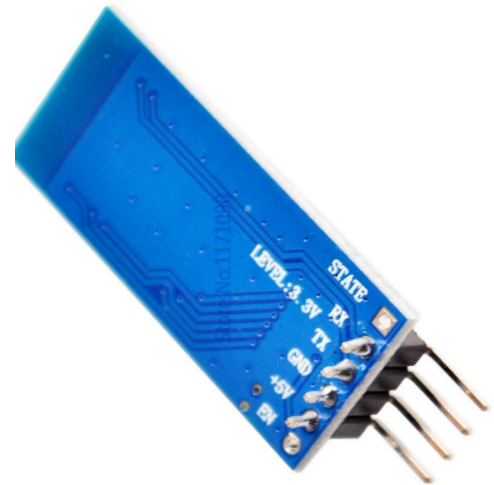


Abbildung 15: Bluetooth-Modul mit HC-06 Unterseite  
(Quelle: roboter-bausatz.de)

In Abbildung 14 ist das verwendete Bluetooth-Modul zu sehen. Bei der grünen aufgelöteten Platine handelt es sich um den HC-06 mit dem CSR BC417BQN (Bluetooth Chip). Der HC-06 unterscheidet sich vom HC-05, indem er nur als Slave fungieren kann. Das bedeutet der HC-06 kann nicht ein Gerät (z. B. Laptop) suchen und sich mit ihm Verbinden. Zu erkennen ist der HC-06 daran, dass nur 4 Pins herausgeführt sind.

Die Pins TX und RX fungieren als UART Schnittstelle.

| PIN        | Beschreibung                                                      |
|------------|-------------------------------------------------------------------|
| RX (3, 3V) | Receive – Eingang UART des Moduls, benötigt einen Spannungsteiler |
| TX         | Transmit – Ausgang UART des Moduls                                |
| GND        | Verbindung zu GND                                                 |
| 5V         | +5V Versorgungsspannung                                           |

#### 3.4.1 Test und konfigurieren mit USB-zu-Seriell-Konverter

Das Bluetooth-Modul wird mit dem USB-zu-Seriell-Konverter verbunden. Es sind keine Widerstände nötig da der Konverter mit 3,3 V arbeitet.

| Bluetooth-Modul | USB-zu-Seriell-Konverter |
|-----------------|--------------------------|
| RX              | TX                       |
| TX              | RX                       |
| GND             | GND                      |
| 5V              | +5V                      |

Die LED des Bluetooth-Moduls sollte blinken, sobald die Spannungsversorgung hergestellt wurde. Dies bedeutet das auf Verbindung gewartet wird.



Nun können die Befehle zum Konfigurieren über UART (siehe 4.2 UART Übertragung, S. 20) mit einer Baud-Rate von 9600 bps übertragen werden. Beim HC-06 kann kein „AT Command Mode“ aktiviert werden. Er befindet sich immer im Kommando-Modus, wenn keine Verbindung besteht was an dem Blinken der LED zu erkennen ist.

| Befehl senden | Beispiel               | Rückgabe vom Modul |
|---------------|------------------------|--------------------|
| AT            |                        | OK                 |
| AT+VERSION    |                        | OKlinvorV1.8       |
| AT+NAME<Name> | AT+NAMEBlinker_Anzeige | OKsetname          |
| AT+PIN<PIN>   | AT+PIN1234             | OKsetPIN           |

Es gibt weitere Befehle (siehe Datenblatt). So ist es zum Beispiel möglich die BAUD-Rate zu verändern, allerdings reicht 9600 bps völlig für den UART gebrauch aus und eine Änderung kann Projekt verwirren.

Schließlich kann man sich mit einem anderen Computer über Bluetooth Verbinden und seriell Daten an den Computer mit der UART Verbindung senden (siehe 4.4 Bluetooth Übertragung, S. 22).

### 3.4.2 Schaltung mit Mikrocontroller

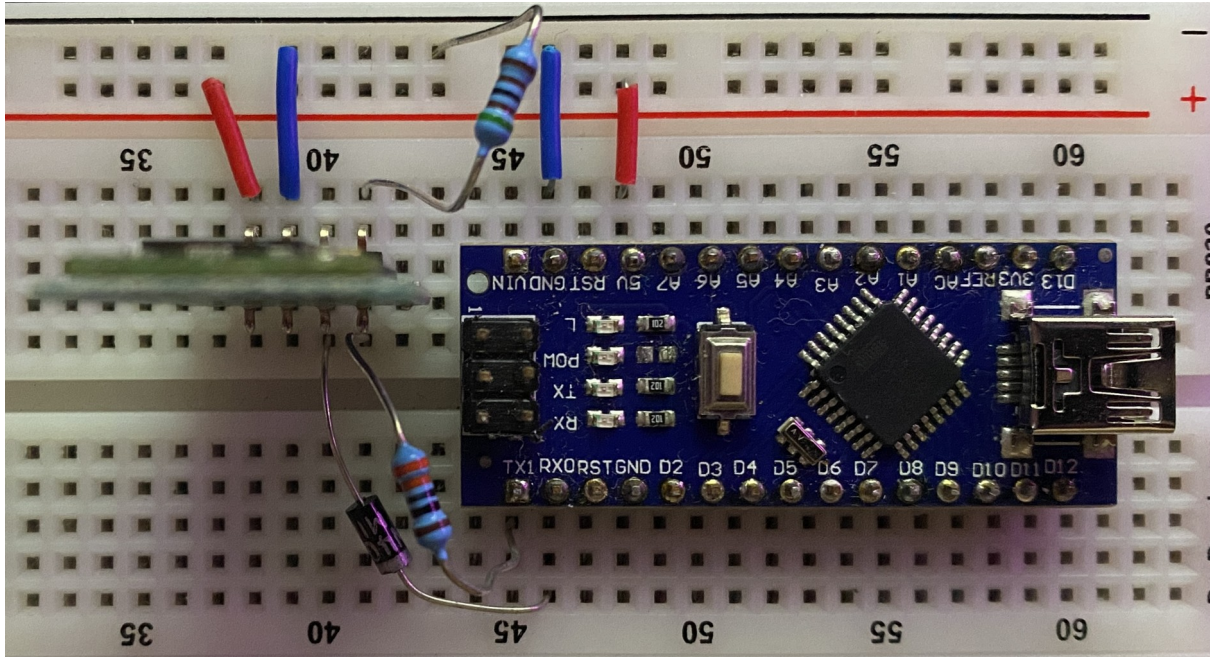
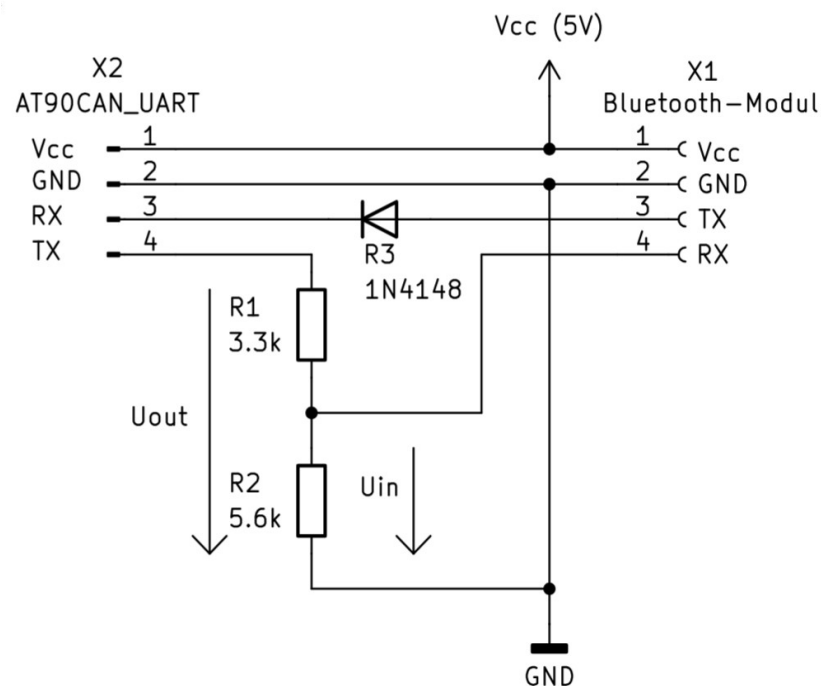


Abbildung 16: Bluetooth-Modul Schaltung mit Arduino



Schaltplanausschnitt 1: Anschluss des Bluetooth-Modul an die Masterplatine

Der TX Pin des Bluetooth-Moduls wird über eine Diode geleitet, da der MCU noch nicht initialisiert sein könnte und damit der RX des MCU High ist und TX Low, dadurch entsteht ein Kurzschluss. Der TX Pin des MCU wird über einen Spannungsteiler angeschlossen da der MCU mit 5V und das Modul mit 3.3 V arbeitet. Im Folgenden wird der Spannungsteiler dimensioniert.

Die Eingangsspannungen beziehen sich auf den CSR BC417BQN.

|                                              |                          |
|----------------------------------------------|--------------------------|
| $U_{out} \in [4,6 \text{ V}; 5,2 \text{ V}]$ | Ausgangsspannung vom MCU |
| $U_{in} \in [2,2 \text{ V}; 4,2 \text{ V}]$  | Eingangsspannung         |
| $U_{in_{max}} = 5,6 \text{ V}$               | Absolute Maximum         |

$$R_1 = 3,3 \text{ k}\Omega \pm 10 \%$$

$$R_2 = 5,6 \text{ k}\Omega \pm 10 \%$$

$$U_{in} = U_{out} \cdot \frac{R_1}{R_2}$$

$$\hat{U}_{in} = 5,2 \text{ V} \cdot \frac{3,3 \text{ k}\Omega \cdot 1,1}{5,6 \text{ k}\Omega \cdot 0,9} \approx 3,75 \text{ V}$$

$$\bar{U}_{in} = 4,8 \text{ V} \cdot \frac{3,3 \text{ k}\Omega}{5,6 \text{ k}\Omega} \approx 2,83 \text{ V}$$

$$\tilde{U}_{in} = 4,6 \text{ V} \cdot \frac{3,3 \text{ k}\Omega \cdot 0,9}{5,6 \text{ k}\Omega \cdot 1,1} \approx 2,22 \text{ V}$$

Schließlich kann man die UART-Ausgabe über Bluetooth auslesen und Daten an den MCU über UART senden (siehe 4.4 Bluetooth Übertragung, S. 22).

**Warnung:** Hochladen über den Bootloader wird nicht funktionieren da ein Reset benötigt wird.

## 4 Grundlagen

In diesem Kapitel werden die Grundlagen zur bedienung der Software erklärt. So das man dieses Projekt weiter entwickeln oder warten kann. Dabei richtet sich diese Anleitung an Linux-Nutzer, einige Programme sind ebenfalls für andere System wie Windows verfügbar.

### 4.1 Kompilieren und Hochladen

Benötigtes Paket:

- `cmake`

CMake und Make sind Werkzeuge, die den Bau von Programmen steuern, ähnlich wie eine IDE (Microchip Studio), nur über die Kommandozeile. CMake erzeugt Baupläne, die das System und den Compiler kennen, und Make führt diese Pläne aus, übersetzt den Code und erstellt die ausführbaren Dateien.

```
$ cmake -B build
```

Dieser Befehl erzeugt in einem neuen Verzeichnis namens „build“ alle notwendigen Dateien, die für den Bau des Projekts benötigt werden. CMake liest die „CMakeLists.txt“ Datei aus dem „src“ Verzeichnis, plant welche Dateien kompiliert werden müssen, und erstellt die Bauanweisungen (Makefile).

```
$ make
```

Make liest die von CMake erzeugten Makefiles und führt die Anweisungen aus. Es kompiliert die Quellcodes in Maschinencode und erstellt daraus die fertigen Programme oder Bibliotheken. Make sorgt dafür, dass nur die geänderten Dateien neu gebaut werden, was Zeit spart.

## 4.2 UART Übertragung

### 4.2.1 Text übertragen

Benötigtes Paket:

- minicom

1. USB-zu-Seriell-Konverter Geräte Schnittstelle finden (z. B. /dev/ttyUSB0 oder /dev/ttyACM0):

```
$ ls /dev/tty*
```

2. Serielle Übertragung öffnen. In diesem Fall mit dem Tool „minicom“ und eine BAUD-Rate von 9600 bps:

```
$ minicom -D /dev/ttyUSB0 -b 9600
```

### 4.2.2 Rohdaten übertragen

Um Binär oder Hexadezimal über UART zu senden kann man nicht *minicom* verwenden. In diesem Beispiel wird das Standard-Linux-Interface genutzt.

1. BAUD-Rate einstellen (z. B. 9600 bps):

```
$ stty -F /dev/ttyUSB0 9600
```

2. Nachricht senden (z. B. für das Cockpit-Display):

```
$ printf 'bL.pic=1\xFF\xFF\xFF' > /dev/ttyUSB0
```

Hierbei ist die Nachricht: „bL.pic=1“ und drei mal 0xFF.

## 4.3 CAN Sniffing (PCAN-USB)

Benötigte Pakete:

- can-utils

Als CAN-Interface für USB wird der PCAN-USB IPEH-002021 von PEAK-System verwendet.

Der D-Sub stecker von allen Modulen ist nach CiA® 106 (siehe Abbildung 3: männlicher CAN D-Sub Anschluss nach CiA® 106 (Quelle: PCAN-USB Anleitung), S. 8)

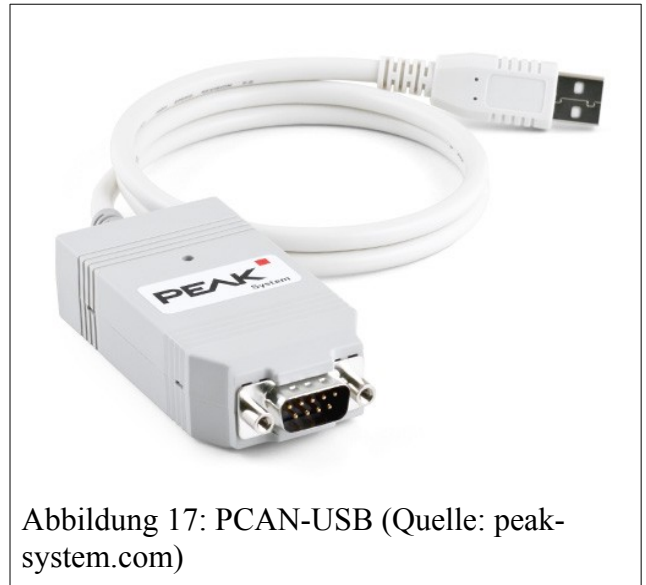


Abbildung 17: PCAN-USB (Quelle: peak-system.com)

### 4.3.1 Empfangen

Der folgende Befehl aktiviert das Netzwerk-Interface `can0` als CAN-Bus-Schnittstelle und setzt dabei die Übertragungsrate auf 125 kbit/s. Dieser Schritt muss nur einmal durchgeführt werden.

```
$ sudo ip link set can0 up type can bitrate 125000
```

Anschließend können Nachrichten des CAN-Busses ausgegeben werden.

```
$ candump can0
```

Mögliche Ausgabe:

```
can0 400 [8] 01 02 02 03 04 05 06 07
```

### 4.3.2 Senden

Beispiel mit ID = 0x4AA und erstes Datenbyte = 0x0F, zweites Datenbyte = 0x55:

```
$ cansend 4AA#0F55
```

### 4.3.3 Fehlerbehebung

Eventuell muss der Treiber von PEAK-System manuell installiert werden.

Ob ein Treiber installiert ist kann man zum Beispiel über die System-Logs herausfinden. Da der Treiber je nach Distribution auch anders als `peak_usb` benannt sein kann, muss man eventuell den gesuchten Eintrag mit `can0`, `pcan` oder `peak` suchen.

```
$ sudo dmesg | grep -i peak_usb
```

Wenn die Ausgabe so ähnlich aussieht ist der Treiber installiert:

```
peak_usb 3-2:1.0: PEAK-System PCAN-USB adapter hwrev 28 serial  
peak_usb 3-2:1.0 can0: attached to PCAN-USB channel 0  
usbcore: registered new interface driver peak_usb
```

Falls der Treiber nicht installiert ist kann man ihn über den Package-Manager des Systems herunterladen oder über die Webseite von PEAK-Systems.



## 4.4 Bluetooth Übertragung

In diesem Kapitel wird erklärt wie man eine Verbindung mit dem Bluetooth-Modul (siehe 3.4 Bluetooth-Modul, S. 15) aufnehmen kann.

**Warnung:** IOS unterstützt keine Verbindung zu SPP (Serial Port Profile), sondern nur BLE (Bluetooth Low Energy). Da es sich hier um SPP handelt, ist keine Verbindung mit dem iPhone oder iPad möglich.

Benötigte Pakete:

- minicom
- bluez
- bluez-deprecated-tools

Die Arch Wiki hat eine ausführliche Erklärung wie man sich mit einem Bluetooth-Gerät verbindet (siehe <https://wiki.archlinux.org/title/Bluetooth>).

Nachdem eine Bluetooth-Verbindung hergestellt wurde, muss sie einer device-Datei gebunden werden:

```
$ sudo rfcomm bind rfcomm0 <MAC Adresse des Bluetooth-Moduls>
```

Anschließend kann eine serielle Übertragung (siehe 4.2 UART Übertragung, S. 20) mit einer BAUD-Rate von 9600 bps hergestellt werden:

```
$ minicom -D /dev/rfcomm0 -b 9600
```

Das Blinken auf dem Bluetooth-Modul sollte aufhören und die Status-LED sollte konstant Leuchten.

## 5 Eidesstattliche Erklärung

Hiermit versichere ich, dass ich die vorliegende Dokumentation selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder nicht veröffentlichten Schriften entnommen wurden, sind als solche kenntlich gemacht.

Ferner versichere ich, dass die vorliegende Arbeit noch nicht in einem anderen Prüfungsverfahren vorgelegen hat.

X

---

Dean Schneider (Auftragnehmer) & Datum

*Dean*

## 6 Anhang

### 6.1 Projekte der Vorgänger

Hiervon sind hauptsächlich JCS-BK\_1025 (Schaltpläne für Lichtanlage), JCS-BK\_1049 (Lichtanlage) und JCS-BK\_1046 (Lenkradknöpfe, Cockpitdisplay) von Bedeutung. [tabelarisch darstellen]

| Jahr | Projektbezeichnung                           | Projektfunktion                                                                                                                             | Auftragnehmer              |
|------|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| 2023 | JCS-BK_1071_GO-KART Blinker_Lenkrad_Anzeige  | Blinkerbetriebnahme, Lenkradanbindung und die Visualisierung per PC/Laptop, mit CAN-BUS Anbindung, Basis ist JCS-BK_MS-063, JCS-BK_MS-123   | Elias Hahne                |
| 2021 | JCS-BK_1046_GO-KART Fahrzeugarmaturenbrett   | MC basiertes Armaturenbrett ist zu fertigen und die Visualisierung per Tablet / Handy, Basis ist JCS-BK_MS-063, JCS-BK_MS-123               | Christian Barth            |
| 2021 | JCS-BK_1049_GO-KART Lichtanlage              | Es sind Front- und Rückleuchten zu erstellen und die Blinker sollen Als Lauflicht ausgeführt werden, Basis ist JCS-BK_MS-060, JCS-BK_MS-130 | Robin Kraus                |
| 2020 | JCS-BK_1034_GO-KART Fahrzeugarmaturenbrett   | MC basiertes Armaturenbrett ist zu fertigen und die Visualisierung per Tablet / Handy, Basis ist JCS-BK_MS-063, JCS-BK_MS-123               | Daniel Plempe              |
| 2020 | JCS-BK_1037_GO-KART Lichtanlage              | Es sind Front- und Rückleuchten zu erstellen und die Blinker sollen Als Lauflicht ausgeführt werden, Basis ist JCS-BK_MS-060, JCS-BK_MS-130 | Daniel Briem               |
| 2019 | JCS-BK_1022_GO-KART Fahrzeugarmaturenbrett   | MC basiertes Armaturenbrett ist zu fertigen und die Visualisierung per Tablet / Handy, Basis ist JCS-BK_MS-063, JCS-BK_MS-123               | Tim Stöhr                  |
| 2019 | JCS-BK_1025_GO-KART Lichtanlage              | Es sind Front- und Rückleuchten zu erstellen und die Blinker sollen Als Lauflicht ausgeführt werden, Basis ist JCS-BK_MS-060, JCS-BK_MS-130 | Christoph Waßmuth          |
| 2018 | JCS-BK_1005_GO-KART Fahrzeugarmaturenbrett   | MC basiertes Armaturenbrett ist zu fertigen und die Visualisierung per Tablet / Handy, Basis ist JCS-BK_MS-063, JCS-BK_MS-123               | Lukas Guse                 |
| 2018 | JCS-BK_1008_GO-KART Lichtanlage              | Es sind Front- und Rückleuchten zu erstellen und die Blinker sollen Als Lauflicht ausgeführt werden, Basis ist JCS-BK_MS-060, JCS-BK_MS-130 | Luis Nolte                 |
| 2017 | JCS-BK_MS-123_GO-KART Fahrzeugarmaturenbrett | MC basiertes Armaturenbrett ist zu fertigen und die Visualisierung per Tablet / Handy, Basis ist JCS-BK_MS-063                              | Nils Kölle                 |
| 2017 | JCS-BK_MS-130_GO-KART Lichtanlage            | Es sind Front- und Rückleuchten zu erstellen und die Blinker sollen Als Lauflicht ausgeführt werden, Basis ist JCS-BK_MS-060                | Johannes Wachs             |
|      | JCS-BK_MS-060_GO-KART Lichtanlage            | Front- und Rückleuchten sind zu fertigen                                                                                                    | Philipp Junge, Jan Schmidt |
|      | JCS-BK_MS-063_GO-KART Fahrzeugarmaturenbrett | JAVA-Tablet und MC Armaturenbrett ist zu fertigen                                                                                           | Patrick Schwarz            |

Tabelle 3: alte Projekte

## 6.2 Tagesziele

| Nr. | Datum                     | Zeit (h) | Tagesziel                                                                                                | Reflexion                                                                                                                                                                                                                             |
|-----|---------------------------|----------|----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | 2025-09-12                | 2,0      | Pflichtenheft Layout erstellen                                                                           | Deckblatt und ungefähres Design fertig. Titelbild muss eingefügt werden.                                                                                                                                                              |
| 2   | 2025-09-22                | 4,5      | Nach Erkältung an Pflichtenheft weiterarbeiten.                                                          | - Lastenheft eingefügt<br>- Projekte der Vorgänger herausgesucht<br>- Bereits vorhandene Elemente und meine Aufgaben herausgeschrieben                                                                                                |
| 3   | 2025-09-25                | 1.5      | Pflichtenheft fortführen, insbesondere alle vorgegeben Tabellen einfügen (Meilensteine, Terminplan, ...) | Terminplan eingefügt                                                                                                                                                                                                                  |
| 4   | 2025-09-26                | 3,0      | Pflichtenheft weiter ausformulieren.                                                                     | 50 Prozent                                                                                                                                                                                                                            |
| 5   | 2025-09-27                | 4,0      | Pflichtenheft Aufgabenstellung definieren. Eventuell Zeitplan anpassen.                                  | Aufgabenstellung weiter definiert und die letzten drei Vorgänger-Projekte analysiert.                                                                                                                                                 |
| 6   | 2025-09-28                | 6,0      | Pflichtenheft erste Version                                                                              | Erste Version fertiggestellt.                                                                                                                                                                                                         |
| 7   | 2025-10-08                | 3,0      | Pflichtenheft korrigieren.                                                                               | Erfolgreich                                                                                                                                                                                                                           |
| 8   | 2025-11-06                | 1,5      | Kegelbahn-Anzeige ansteuern                                                                              | Schaltplan analysiert, Anschlüsse festgestellt                                                                                                                                                                                        |
| 9   | 2025-11-07                | 1,5      | - Kegelbahn-Anzeige ansteuern<br>- OLED Display ansteuern                                                | Erfolgreich angesteuert                                                                                                                                                                                                               |
| 10  | 2025-11-08                | 3,0      | Bluetooth Modul ansteuern                                                                                | Kann mit USB angesteuert werden.                                                                                                                                                                                                      |
| 11  | 2025-11-09                | 6,0      | Bluetooth Modul ansteuern                                                                                | Kann vollständig angesteuert werden und vollständig dokumentiert.                                                                                                                                                                     |
| 12  | 2025-11-10                | 6,0      | Display testen und ansteuern                                                                             | Das Display wurde mit 14V gebraten. Ich habe ein Ersatzdisplay bekommen. Allerdings habe ich heute größtenteils den anderen geholfen und bin daher nicht weiter gekommen.                                                             |
| 13  | 2025-11-11                | 6,0      | Hausaufgaben zu den Kegelbahnanzeigen bearbeiten                                                         | Punkteanzeige fertig                                                                                                                                                                                                                  |
| 14  | 2025-11-12                | 6,0      | Wurfanzeige für die Kegelbahn                                                                            | Muss nur noch hochgeladen werden.                                                                                                                                                                                                     |
| 15  | 2025-11-13                | 6,0      | - Hausaufgabe zur Kegelbahn fertigstellen<br>- kaputtes Display messen<br>- AT90CAN Platine analysieren  | - Hausaufgabe fertig<br>- AT90CAN Platine wurde analysiert, allerdings funktioniert Hochladen aufgrund von avrdude noch nicht. Ich muss noch Feststellen, welchen Typ der Programmierer hat.<br>- kaputtes Display messe ich zu Hause |
| 16  | 2025-11-27 bis 2025-11-29 | 6,0      | Hausaufgabe TI_85_Arduino_CAN-BUS_Test nachholen. Testaufbau für CAN-Bus mit dem MCP2515 Modul.          | Ich brauche ein zweites MCP2515 (und TJA1050) Modul um einen richtigen Test durchführen zu können. Ich kann also erst in der Schule weiterarbeiten.                                                                                   |
| 17  | 03.12.25                  | 4,0      | OLED Display ansteuern                                                                                   | Funktioniert mit Bibliothek. Außerdem Datenblatt analysiert, um die Einstellungen zu verstehen und eventuell selbst die Befehle zu senden.                                                                                            |
| 18  | 05.12.25                  | 1,5      | CAN Bus Test: Senden und Empfangen                                                                       | CAN Bus Analyzer (PCAN) mit Software zum laufen bringen. Nicht abgeschlossen                                                                                                                                                          |
| 19  | 05.12.25                  | 3,0      | CAN Bus Test: Senden und Empfangen                                                                       | Senden und Empfangen über ein CAN USB Interface funktioniert                                                                                                                                                                          |
| 20  | 12.12.25                  | 3,0      | OLED Display ansteuern                                                                                   | An PWM und ADC gearbeitet                                                                                                                                                                                                             |
| 21  | 19.12.25                  | 3,0      | CAN-Bus senden                                                                                           | OLED-Display, PWM, Analog mit C fertig<br>SPI funktioniert nicht mit ATmega328p                                                                                                                                                       |
| 22  | 08.01.26                  | 1,5      | CAN-Bus-Nachricht senden                                                                                 | CAN-Bus-Nachricht senden und empfangen funktioniert.                                                                                                                                                                                  |
| 23  | 09.01.26                  | 5,0      | CAN Nachrichten auf OLED anzeigen.                                                                       | Erfolgreich. Die Nachrichten werden fortlaufend angezeigt.                                                                                                                                                                            |
| 24  | 15.01.26                  | 1,5      | Nextion Display ansteuern.                                                                               | Das Nextion Display mit UART ansteuern und mit den anderen Bluetooth getestet.                                                                                                                                                        |
| 25  | 16.01.26                  | 3,0      | Cockpit Display angesteuert                                                                              | Programm zum Ansteuern geschrieben                                                                                                                                                                                                    |
| 26  | 19.01.26                  | 3,0      | Programm testen und Dokumentation schreiben über CAN-Bus, Display und Cockpit-Display.                   | Programm getestet                                                                                                                                                                                                                     |

Tabelle 4: Tagesziele Teil 1

| <b>Nr.</b> | <b>Datum</b> | <b>Zeit (h)</b> | <b>Tagesziel</b>                                                   | <b>Reflexion</b>                                                                                                                                                  |
|------------|--------------|-----------------|--------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 27         | 20.01.26     | 3,0             | Dokumentation schreiben über CAN-Bus, Display und Cockpit-Display. | Schaltplan für Bluetooth, cmake Erklärung, CAN-Sniffing                                                                                                           |
| 28         | 22.01.26     | 3,0             | Blinker Modul ansteuern                                            | Blinker Modul muss neu programmiert werden, da das letzte Projekt (JCS-BK_1071) den Microcontroller mit einen Testprogramm programmiert hat.<br>CAN-Bus-Steuerung |
| 29         | 22.01.26     | 4,0             | Dokumentation weiter schreiben                                     | allgemeine Verbesserungen<br>Cockpit-Display Kapitel erste Version<br>OLED-Display Kapitel erste Version                                                          |
| 30         | 23.01.26     | 7,0             | Blinker Mikrocontroller testen<br>Dokumentation Draft 2            | Blinker LEDs getestet, als Nächstes muss das Programm zum Steuern über CAN-Bus erstellt werden.<br>Draft 2 fertiggestellt                                         |

Tabelle 5: Tagesziele Teil 2

### 6.3 Meilensteine

| Start-datum | Ziel-datum | Zeit (h) | Thema (Aufgabe)                                                                                                                                                         |
|-------------|------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 01.09.25    | 11.09.25   |          | Festlegen des Themas und der Projektteams (Informationssammlung, Pflichtenheft zur Aufgabenfixierung, Aufgabenverteilung im Team und Anfertigung des Detailterminplans) |
| 12.09.25    | 08.10.25   | 18       | <b><u>Abgabe des Pflichtenhefts</u></b>                                                                                                                                 |
| 08.10.25    | 30.11.25   | 34       | Schaltplanerstellung / Programmierung von Modulen / Testaufbau / Modultests                                                                                             |
| 08.10.25    | 30.11.25   | 11       | <b><u>Abgabe der Draft 1 - Dokumentationsentwurf</u></b>                                                                                                                |
| 31.11.25    | 23.01.26   | 5        | Statusermittlung / Terminplanprüfung                                                                                                                                    |
| 31.11.25    | 23.01.26   | 7        | Schaltplan- und Layouterstellung / Programmierung von Modulen<br>Testaufbau / Modultests / Dokumentation                                                                |
| 31.11.25    | 23.01.26   | 15       | Programmtests, Dokumentation, Fehleranalyse, Fehlerbeseitigung                                                                                                          |
| 31.11.25    | 23.01.26   | 20       | Abgabe der Draft2-Dokumentation 7:35 keine Verlängerung                                                                                                                 |
|             |            |          | Programmtests, Dokumentation, Fehleranalyse, Fehlerbeseitigung                                                                                                          |
|             |            |          | Präsentation des Projekts Weiterentwicklung                                                                                                                             |
|             |            |          | <b>Präsentation der Ergebnisse als Impress-Vortrag</b>                                                                                                                  |
|             |            |          | <b><u>Präsentation der Zwischenstände am Tag der offenen Tür</u></b>                                                                                                    |
|             |            |          | <b>Abgabe des Projekts mit Abschluss-Dokumentation 24:00</b>                                                                                                            |
|             |            |          | <b>Leistungsüberprüfung / Produktabnahme</b>                                                                                                                            |

## 6.4 Terminplan

[illegible]

Tabelle 6: Terminplan 01.09. - 19.09.2025





Tabelle 8: Terminplan 08.12.2025 - 25.01.2026





## 6.5 Abbildungsverzeichnis

|                                                                                                                               |    |
|-------------------------------------------------------------------------------------------------------------------------------|----|
| Abbildung 1: Kommunikationsdiagramm.....                                                                                      | 4  |
| Abbildung 2: Cockpit-Display (NX4024K032).....                                                                                | 6  |
| Abbildung 3: männlicher CAN D-Sub Anschluss nach CiA® 106 (Quelle: PCAN-USB Anleitung).....                                   | 8  |
| Abbildung 4: Debug-Display (OLED-Display mit SSD1306).....                                                                    | 9  |
| Abbildung 5: Stecker zum Programmieren des ATmega16M1.....                                                                    | 10 |
| Abbildung 6: Oberseite des ATmega16M1.....                                                                                    | 10 |
| Abbildung 7: Unterseite des ATmega16M1.....                                                                                   | 10 |
| Abbildung 8: Anschlüsse des AT90CAN32 (Quelle: Schaltplanausschnitt von MS-001).....                                          | 11 |
| Abbildung 9: Masterplatine mit AT90CAN32 (MS-001).....                                                                        | 11 |
| Abbildung 10: ISP Pinbelegung (Quelle: wikipedia.org von osiixy erstellt).....                                                | 12 |
| Abbildung 11: Masterplatine mit serieller und ISP Verbindung.....                                                             | 13 |
| Abbildung 12: Nextion Display (NX8048K070_011C).....                                                                          | 14 |
| Abbildung 13: Ersatz Bauteil suche für Spannungsregler ( <a href="https://smd.yooneed.one">https://smd.yooneed.one</a> )..... | 14 |
| Abbildung 14: Bluetooth-Modul mit HC-06 Oberseite (Quelle: roboter-bausatz.de).....                                           | 15 |
| Abbildung 15: Bluetooth-Modul mit HC-06 Unterseite (Quelle: roboter-bausatz.de).....                                          | 15 |
| Abbildung 16: Bluetooth-Modul Schaltung mit Arduino.....                                                                      | 17 |
| Abbildung 17: PCAN-USB (Quelle: peak-system.com).....                                                                         | 21 |

## 6.6 Schaltplanausschnitte

|                                                                                 |    |
|---------------------------------------------------------------------------------|----|
| Schaltplanausschnitt 1: Anschluss des Bluetooth-Modul an die Masterplatine..... | 17 |
|---------------------------------------------------------------------------------|----|

## 6.7 Tabellenverzeichnis

|                                                    |    |
|----------------------------------------------------|----|
| Tabelle 1: Team.....                               | 4  |
| Tabelle 2: Befehle für das Cockpit-Display.....    | 7  |
| Tabelle 3: alte Projekte.....                      | 24 |
| Tabelle 4: Tagesziele Teil 1.....                  | 25 |
| Tabelle 5: Tagesziele Teil 2.....                  | 26 |
| Tabelle 6: Terminplan 01.09. - 19.09.2025.....     | 28 |
| Tabelle 7: Terminplan 20.10. - 07.12.2025.....     | 29 |
| Tabelle 8: Terminplan 08.12.2025 - 25.01.2026..... | 30 |
| Tabelle 9: Terminplan 26.01. - 15.03.2026.....     | 31 |
| Tabelle 10: Terminplan 16.03. - 10.05.2026.....    | 32 |